Basic (guess token counts, then check) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `token_count_guessing_practice.md`: every cost and every context limit is counted in tokens, not words. Guessing first and then checking shows where the quick rule works and where it breaks. **If not:** you'd plan prompts and budgets with a rule you've never tested.

Solution

Story: two quick estimates next to the real count, so you can see which one fails on which kind of text — and why. **If not:** you'd keep one rule of thumb and never notice its blind spots.

The same table, with both estimation methods shown side by side, and the actual mechanism explained for each surprising row:

Sentence	Char ÷ 4	Words × 1.3	Actual	Why the gap (if any)
"Hi."	~1	~1-2	1-2	No gap — trivially short, matches the rule directly.
"The cat sat on the mat."	~7	~8	6-8	Both estimates land close — this is the "average English sentence" the rule of thumb is calibrated for.
"Unbelievably, the antidisestablishmentarianism debate continued."	~13	~5	9-11	The word-count method fails badly here — one very long, rare word ("antidisestablishmentarianism") splits into many small sub-word tokens, but only counted as "1 word" in the word-based guess.
`print('hello world')`	~5	~3	5-7	Code has meaningful punctuation (`(` , `'` , `)`) that each often becomes its own token — word-counting code badly undercounts.

Sentence	Char ÷ 4	Words × 1.3	Actual	Why the gap (if any)
"میں آج بازار جا ر□ا □وں" (Urdu)	varies	~7	often 2-3× higher	Non-Latin scripts were far less represented in training data used to build the token vocabulary, so the same *meaning* takes noticeably more tokens to express.

The actual mechanism, briefly: the tokenizer's vocabulary was built by finding the most common recurring chunks in a mostly-English, mostly-prose training set. Anything that looks like that training data (common English words) compresses efficiently into few tokens. Anything that doesn't (rare words, code syntax, other scripts) gets broken into smaller, less efficient pieces — because those exact chunks weren't common enough to earn their own single token.

Which estimate should you actually use, day to day? For a one-off "will this fit / roughly what will this cost" check, chars ÷ 4 in your head is enough — precision below about 20% doesn't change any real decision. For a system prompt or any other text resent unchanged on every call of a running feature, don't estimate at all: verify with `tiktoken` that what you're sending is actually what you think it is.